

BTW-Cargo

Digital Twin d'un port à conteneurs

Une application de gestion logistique maritime qui simule la vie d'un port à conteneurs en temps réel, permettant aux opérateurs de valider les chargements et aux administrateurs de superviser l'ensemble des flux (navires, camions, trains).

Présenté par : **Matteo Ventura**

Projet d'entraînement

Date : 23/10/2026

Le Problème
La Solution : BTW-Cargo
Les Fonctionnalités Clés
1. Le Dashboard Administrateur : Supervision en temps réel
Le dashboard admin offre une vue d'ensemble de l'activité du port. On y retrouve le taux de remplissage global, les statistiques de la flotte (navires en mer, à quai, en maintenance), un graphique de répartition des conteneurs et un fil d'actualité des derniers mouvements.
2. L'Inventaire des Conteneurs : Recherche et Filtres Avancés
Avec plus de 70 000 conteneurs en base de données, l'inventaire utilise une pagination serveur pour rester performant. Les opérateurs peuvent filtrer par type, destination, couleur ou statut, et les administrateurs peuvent modifier la destination ou supprimer un conteneur.
3. Le Tour de Contrôle : Messagerie en temps réel
Un système de communication descendant permet à l'administrateur de diffuser des instructions (Info, Warning, Urgent) aux opérateurs. Ces messages s'affichent sous forme de bandeaux d'alerte sur le dashboard des opérateurs et sont historisés.
L'Architecture Technique
a) Le Flux de Données
b) Les Choix Technologiques Forts
La Modélisation des Données (MCD/MLD)
a) Le Modèle Conceptuel de Données (MCD)
b) Les Relations
c) Le Modèle Logique de Données (MLD)
La Structure du Projet
L'application est découpée en modules logiques pour séparer les responsabilités.
1. Le moteur de simulation (dossier /simulation)
C'est le cœur de l'application. Il est composé de plusieurs fichiers :
● logistics.ts : Les constantes et les générateurs de données (codes, types, couleurs).
● shipOperations.ts : La gestion du cycle de vie des navires (arrivées, chargements, départs).
● truckOperations.ts : La gestion des flux de camions entrants et sortants.
● trainOperations.ts : La gestion des trains (horaires fixes et trains d'urgence).
● maintenance.ts : La rotation de maintenance de la flotte.
2. L'interface utilisateur (dossier /src/app)
● L'architecture utilise le App Router de Next.js 15.
● /dashboard : Toutes les pages protégées (admin, containers, ships, users, messages, profile).
● /login : La page d'authentification.
● /actions : Les Server Actions (le "cerveau" du backend, sécurisé et typé).
3. Les composants (dossier /src/components)
Tests
Un projet professionnel ne peut pas être livré sans une stratégie de test claire et une sécurité robuste.
🧪 Une Suite de Tests Automatisés (Vitest)
Pour garantir la fiabilité du code, 12 tests ont été écrits et validés :

● Tests Unitaires (5) : Vérification des générateurs de codes de conteneurs et des profils de navires.

● Tests d'Intégration (1) : Création d'un conteneur dans une base de données isolée (btw_cargo_test).

● Tests de Sécurité (6) : Vérification que les opérateurs ne peuvent pas effectuer d'actions admin (RBAC) et que le Rate Limiting bloque après 5 échecs de connexion.

(Capture d'écran des tests réalisés avec Vitest : Résultat : 12 tests passés, 100% de réussite.)

Sécurité

🔒 Une Sécurité de Niveau Production

● Authentification : Jetons JWT stockés dans des cookies HttpOnly (inaccessibles par JavaScript).

● Mots de passe : Hachés avec Bcrypt (impossible à décrypter).

● Validation : Toutes les données entrantes sont validées avec Zod avant d'être traitées.

● Protection XSS : Les messages du Tour de Contrôle sont nettoyés (sanitization) avant affichage.

● En-têtes de sécurité : X-Frame-Options et X-Content-Type-Options sont configurés via Next.js.

(Capture d'écran du la securite implémentée au niveau des “headers”)

Conclusion & Perspectives

Ce que ce projet m'a apporté

Les Perspectives d'Évolution

Le Problème

La gestion d'un port à conteneurs est un défi logistique majeur. Les informations sont souvent dispersées, les flux (navires, camions, trains) sont difficiles à coordonner, et les décisions doivent être prises en temps réel pour éviter la saturation ou le chômage des terminaux. Sans un outil centralisé, les opérateurs perdent du temps et les administrateurs naviguent à vue.

La Solution : BTW-Cargo

BTW-Cargo est une application web centralisée qui agit comme un véritable poste de commandement. Elle résout ces problèmes en offrant :

- Une supervision en temps réel : Les administrateurs visualisent l'état du port (taux de remplissage, navires à quai, en mer) sur un dashboard dynamique.
- Un espace opérateur dédié : Les agents de quai disposent d'une file d'attente claire pour valider les chargements en un clic.
- Un moteur de simulation autonome (Digital Twin) : Le port "vit" tout seul grâce à un CRON qui simule les arrivées de camions, les départs de trains et les rotations de navires.

En résumé, BTW-Cargo transforme une gestion complexe et dispersée en un flux de travail fluide, sécurisé et visuel.

Les Fonctionnalités Clés

1. Le Dashboard Administrateur : Supervision en temps réel

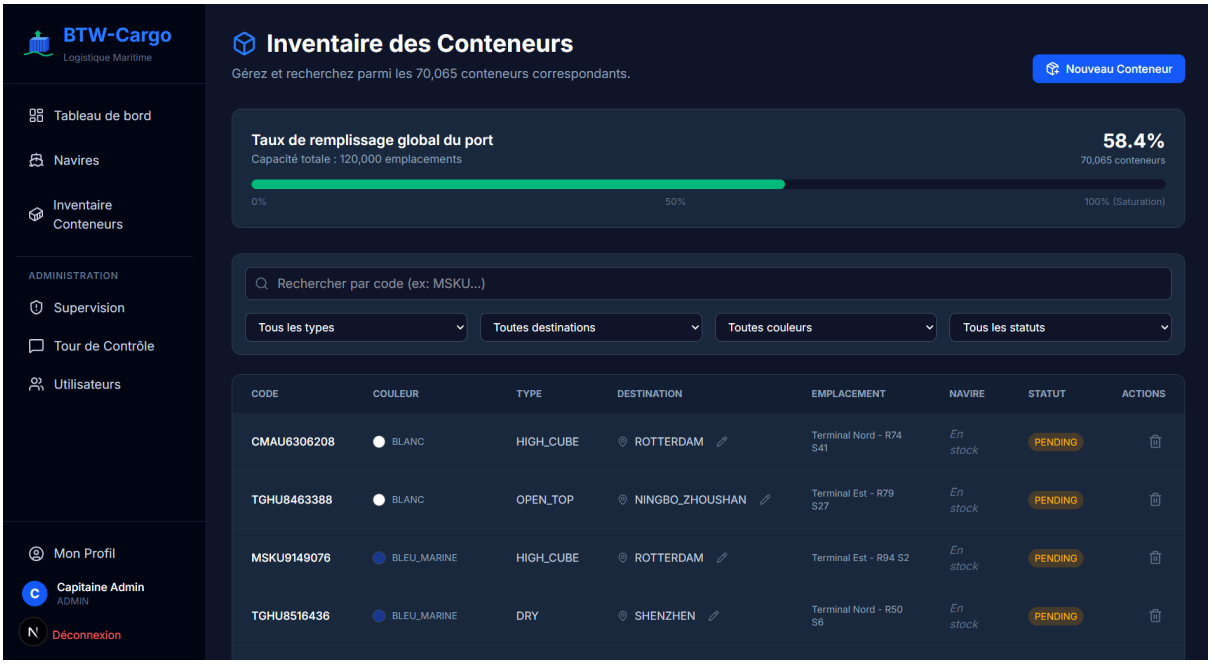
Le dashboard admin offre une vue d'ensemble de l'activité du port. On y retrouve le taux de remplissage global, les statistiques de la flotte (navires en mer, à quai, en maintenance), un graphique de répartition des conteneurs et un fil d'actualité des derniers mouvements.



(Capture d'écran de la page “Dashboard” en tant qu’administrateur)

2. L'Inventaire des Conteneurs : Recherche et Filtres Avancés

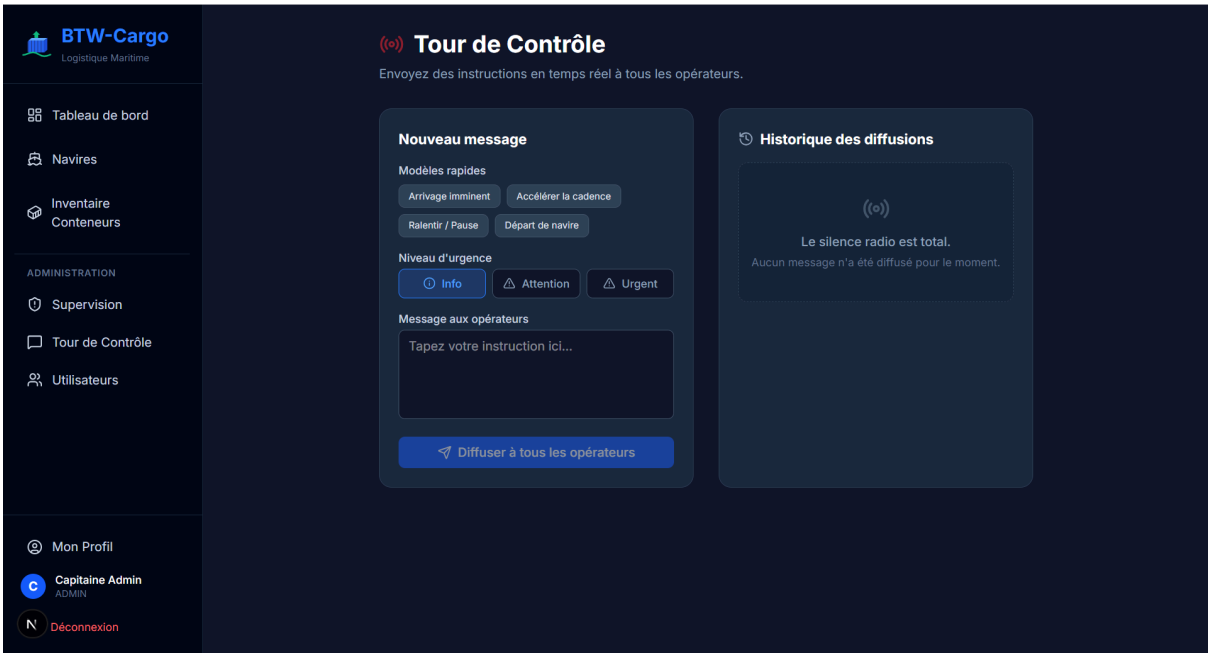
Avec plus de 70 000 conteneurs en base de données, l'inventaire utilise une pagination serveur pour rester performant. Les opérateurs peuvent filtrer par type, destination, couleur ou statut, et les administrateurs peuvent modifier la destination ou supprimer un conteneur.



(Capture d'écran de la page “Inventaire” qui reste inchangée que ca soit pour un opérateur ou pour un admin)

3. Le Tour de Contrôle : Messagerie en temps réel

Un système de communication descendant permet à l'administrateur de diffuser des instructions (Info, Warning, Urgent) aux opérateurs. Ces messages s'affichent sous forme de bandeaux d'alerte sur le dashboard des opérateurs et sont historisés.



(Capture d'écran de la page “Tour de Contrôle” en tant qu’administrateur)

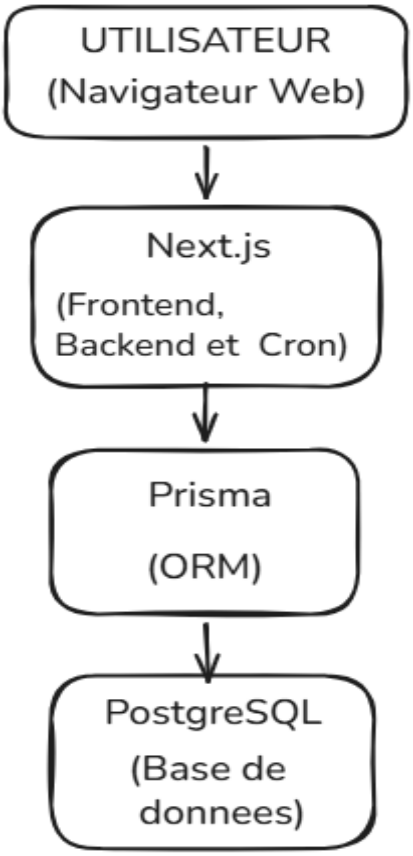
L'Architecture Technique

BTW-Cargo repose sur une architecture full-stack moderne, pensée pour la performance et le temps réel.

a) Le Flux de Données

Voici comment circule l'information au sein de l'application :

1. L'Utilisateur (navigateur web) accède à l'interface Next.js.
2. Next.js gère à la fois le Frontend (affichage) et le Backend (Server Actions, API Routes).
3. Prisma (l'ORM) fait le lien entre le code et la base de données.
4. PostgreSQL stocke les données métier (utilisateurs, navires, conteneurs).
5. En arrière-plan, un CRON (Node-Cron) simule la vie du port en continu.



(Schéma du flux de données)

b) Les Choix Technologiques Forts

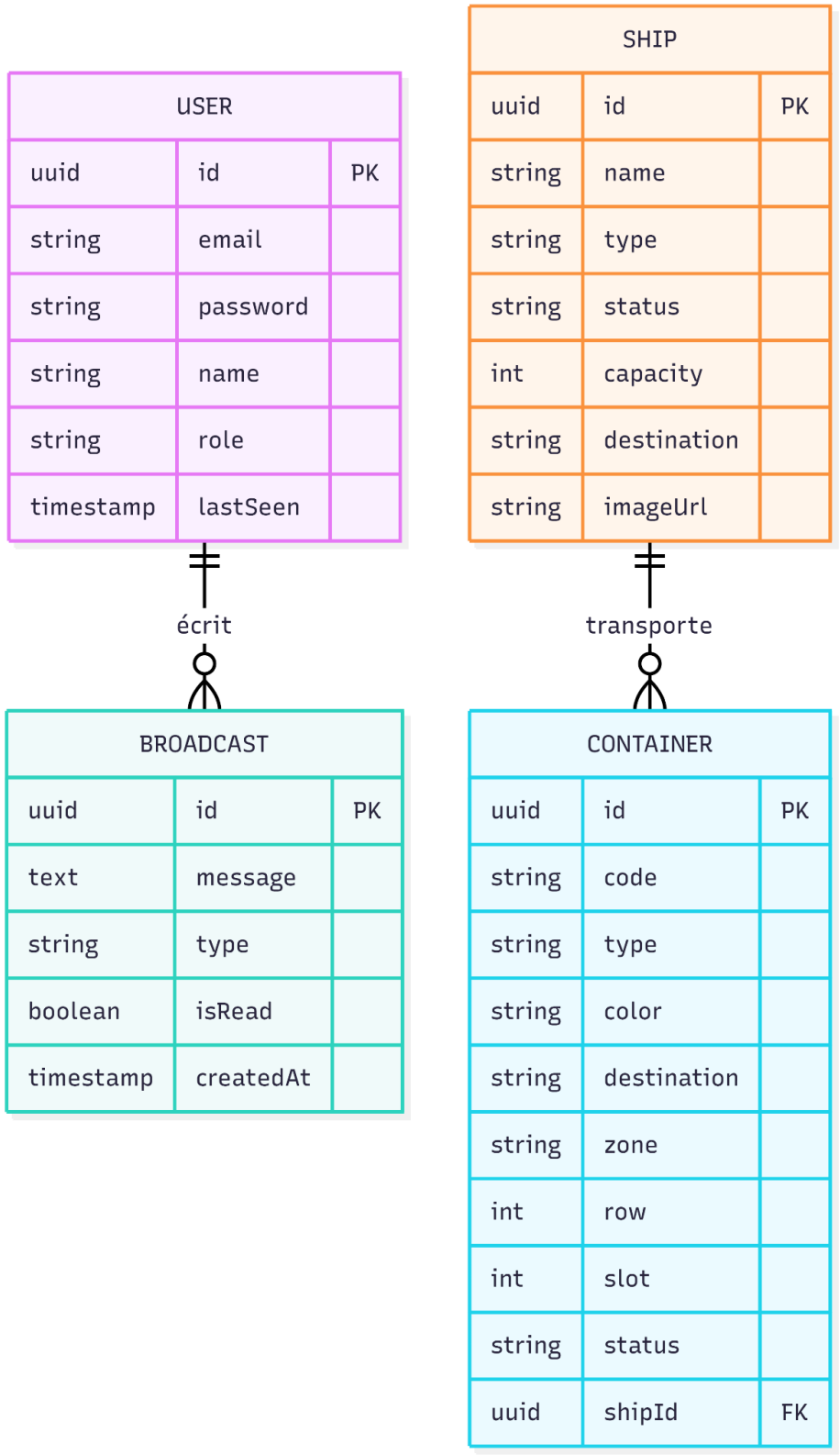
- Next.js 15 (App Router) : Pour un rendu ultra-rapide (Server Components) et des Server Actions sécurisées.
- PostgreSQL + Prisma : Une base de données relationnelle robuste avec un typage TypeScript de bout en bout.
- Le Moteur de Simulation (CRON) : Un "Digital Twin" qui tourne toutes les 30 secondes pour simuler l'arrivée des navires, le passage des camions et l'évacuation par train.

La Modélisation des Données (MCD/MLD)

La structure de la base de données a été pensée pour être à la fois simple, robuste et évolutive. Elle repose sur 4 entités principales.

a) Le Modèle Conceptuel de Données (MCD)

- USER : Représente les utilisateurs de l'application (Admin ou Opérateur).
- SHIP : Représente les navires de la flotte (avec leur type, capacité et statut).
- CONTAINER : Représente les conteneurs (avec leur code unique, type, couleur et emplacement).
- BROADCAST : Représente les messages diffusés par le Tour de Contrôle.



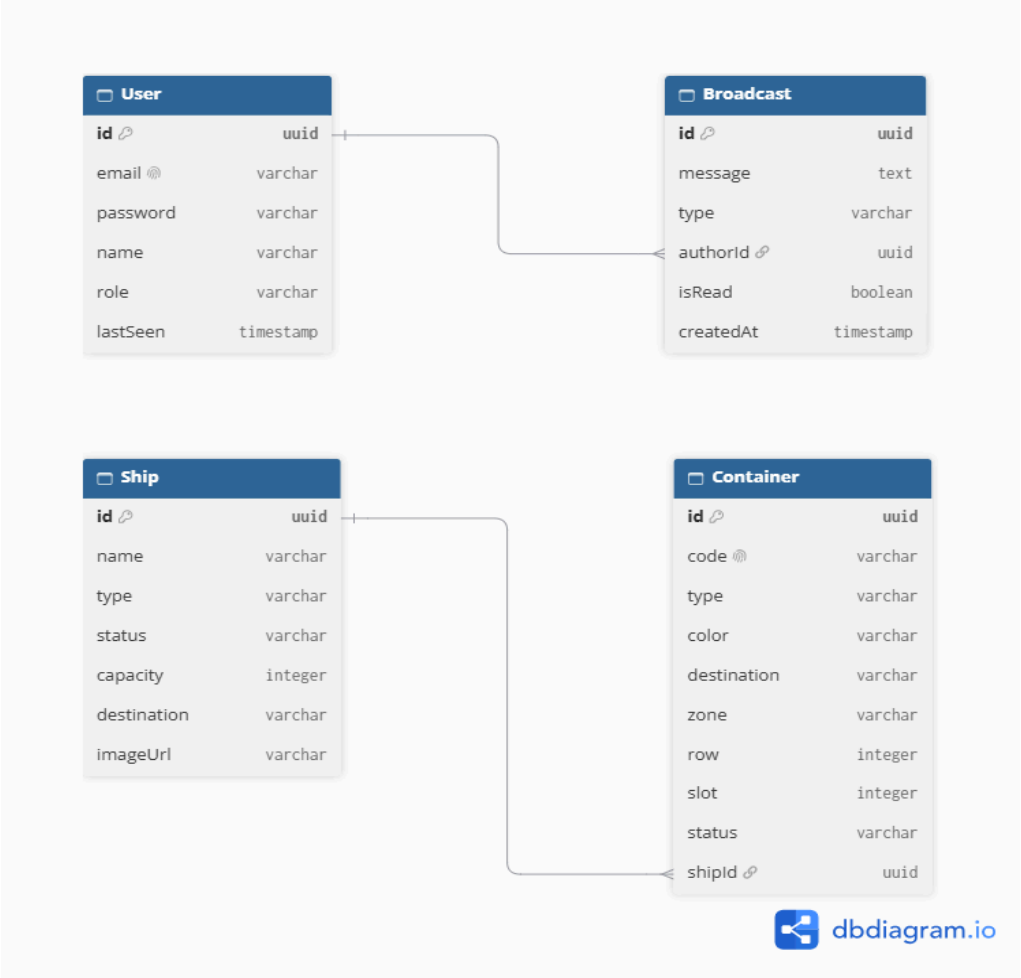
(Diagramme Modèle Conceptuel de Données)

b) Les Relations

- Un utilisateur admin peut envoyer plusieurs messages (BROADCAST).
- Un navire peut contenir plusieurs conteneurs (CONTAINER).
- Un conteneur peut être sur un seul navire (ou aucun s'il est en stock).

c) Le Modèle Logique de Données (MLD)

Voici comment les tables sont structurées physiquement dans PostgreSQL :



(Diagramme du Modèle Logique de Données)

La Structure du Projet

L'application est découpée en modules logiques pour séparer les responsabilités.

1. Le moteur de simulation (dossier `/simulation`)

C'est le cœur de l'application. Il est composé de plusieurs fichiers :

- `logistics.ts :` Les constantes et les générateurs de données (codes, types, couleurs).
- `shipOperations.ts :` La gestion du cycle de vie des navires (arrivées, chargements, départs).
- `truckOperations.ts :` La gestion des flux de camions entrants et sortants.
- `trainOperations.ts :` La gestion des trains (horaires fixes et trains d'urgence).
- `maintenance.ts :` La rotation de maintenance de la flotte.

2. L'interface utilisateur (dossier `/src/app`)

- L'architecture utilise le App Router de Next.js 15.
- `/dashboard :` Toutes les pages protégées (admin, containers, ships, users, messages, profile).
- `/login :` La page d'authentification.
- `/actions :` Les Server Actions (le "cerveau" du backend, sécurisé et typé).
- `/api :` Les routes API (statistiques en temps réel).

3. Les composants (dossier `/src/components`)

- Tous les éléments réutilisables de l'interface (les tableaux, les modales, le logo, les bandeaux d'alerte).

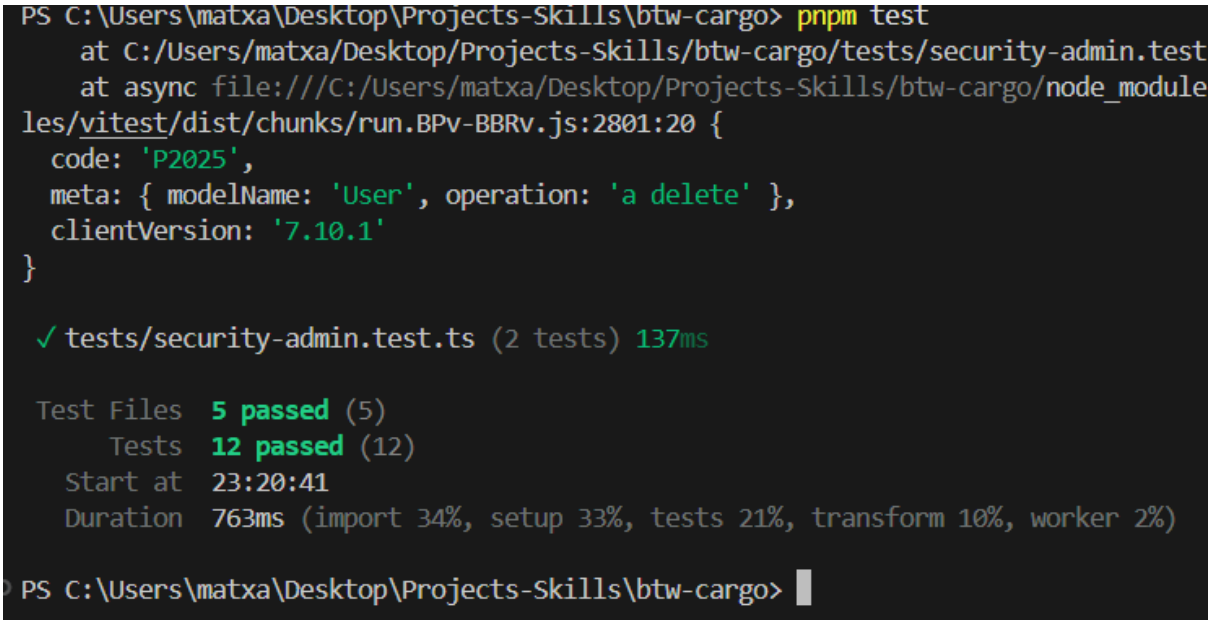
Tests

Un projet professionnel ne peut pas être livré sans une stratégie de test claire et une sécurité robuste.

🧪 Une Suite de Tests Automatisés (Vitest)

Pour garantir la fiabilité du code, 12 tests ont été écrits et validés :

- Tests Unitaires (5) : Vérification des générateurs de codes de conteneurs et des profils de navires.
- Tests d'Intégration (1) : Création d'un conteneur dans une base de données isolée (btw_cargo_test).
- Tests de Sécurité (6) : Vérification que les opérateurs ne peuvent pas effectuer d'actions admin (RBAC) et que le Rate Limiting bloque après 5 échecs de connexion.



(Capture d'écran des tests réalisés avec Vitest : Résultat : 12 tests passés, 100% de réussite.)

Sécurité

🔒 Une Sécurité de Niveau Production

- Authentification : Jetons JWT stockés dans des cookies HttpOnly (inaccessibles par JavaScript).
- Mots de passe : Hachés avec Bcrypt (impossible à décrypter).
- Validation : Toutes les données entrantes sont validées avec Zod avant d'être traitées.
- Protection XSS : Les messages du Tour de Contrôle sont nettoyés (sanitization) avant affichage.
- En-têtes de sécurité : X-Frame-Options et X-Content-Type-Options sont configurés via Next.js.



```
/* Les en-têtes de sécurité HTTP */
async headers() {
  return [
    {
      source: '/(.*)', // S'applique à toutes les routes
      headers: [
        {
          key: 'X-Frame-Options',
          value: 'DENY',
        },
        {
          key: 'X-Content-Type-Options',
          value: 'nosniff',
        },
        {
          key: 'Referrer-Policy',
          value: 'origin-when-cross-origin',
        },
      ],
    },
  ];
},
};
```

(Capture d'écran du la securite implémentée au niveau des “headers”)

Conclusion & Perspectives

Ce que ce projet m'a apporté

La conception de BTW-Cargo m'a permis de monter en compétences sur des technologies modernes et exigeantes (Next.js 15, Prisma, PostgreSQL). J'ai particulièrement appris à :

- Concevoir une architecture full-stack sécurisée avec Next.js (JWT, RBAC) et la tester.
- Gérer des flux de données complexes en temps réel avec un moteur de simulation (CRON).
- Rédiger une documentation technique complète (README, MCD/MLD) et mettre en place une suite de tests automatisés.

Les Perspectives d'Évolution

Cette version est une base solide, mais plusieurs améliorations sont déjà identifiées :

1. Temps réel avancé : Remplacer le "polling" actuel par des WebSockets pour une mise à jour instantanée des dashboards.
2. Notifications Push : Alerter les opérateurs directement sur leur smartphone (via PWA) lors d'un message urgent du Tour de Contrôle.
3. Intelligence Artificielle : Prédire les futurs pics de saturation du port en analysant les historiques de flux.